EXPERIMENT 8

Implement a variational autoencoder (VAE) to generate new images from a given dataset. Train the VAE to learn the latent representation of images and generate new samples from the learned distribution.

```
import tensorflow as tf
from tensorflow.keras import layers, Model
import numpy as np
import matplotlib.pyplot as plt

# Load MNIST dataset
(x_train, _), (x_test, _) = tf.keras.datasets.mnist.load_data()

x_train = x_train[:5000].astype("float32") / 255.0
x_test = x_test[:1000].astype("float32") / 255.0

x_train = x_train.reshape(-1, 784)
x_test = x_test.reshape(-1, 784)

latent_dim = 2

# Encoder
inputs = layers.Input(shape=(784,))
x = layers.Dense(256, activation="relu")(inputs)

z_mean = layers.Dense(latent_dim)(x)
z_log_var = layers.Dense(latent_dim)(x)

# Sampling layer
def sampling(args):
    mean, log_var = args
    epsilon = tf.random.normal(shape=tf.shape(mean))
    return mean + tf.exp(0.5 * log_var) * epsilon

z = layers.Lambda(sampling)([z_mean, z_log_var])

encoder = Model(inputs, [z_mean, z_log_var, z])

# Decoder
latent_inputs = layers.Input(shape=(latent_dim,))
x = layers.Dense(256, activation="relu")(latent_inputs)
outputs = layers.Dense(784, activation="sigmoid")(x)

decoder = Model(latent_inputs, outputs)

# VAE model
class VAE(Model):
    def __init__(self, encoder, decoder):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder

    def train_step(self, data):
        if isinstance(data, tuple):
            data = data[0]

        with tf.GradientTape() as tape:
            mean, log_var, z = self.encoder(data)
            reconstruction = self.decoder(z)

            recon_loss = tf.reduce_mean(
                tf.reduce_sum(
                    tf.keras.losses.binary_crossentropy(
                        data, reconstruction
                    ), axis=-1
                )
            )

            kl_loss = -0.5 * tf.reduce_mean(
                tf.reduce_sum(
                    1 + log_var - tf.square(mean) - tf.exp(log_var),
                    axis=1
                )
            )

            total_loss = recon_loss + kl_loss

        gradients = tape.gradient(
```

```
            total_loss, self.trainable_variables
        )

        self.optimizer.apply_gradients(
            zip(gradients, self.trainable_variables)
        )

        return {"loss": total_loss}

vae = VAE(encoder, decoder)
vae.compile(optimizer="adam")

# Train VAE
print("Training VAE...")
vae.fit(x_train, epochs=5, batch_size=64)

# Generate new images
random_latent = np.random.normal(size=(10, latent_dim)).astype("float32")
generated = decoder.predict(random_latent)

# Display generated images
plt.figure(figsize=(10, 4))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(generated[i].reshape(28, 28), cmap="gray")
    plt.axis("off")

plt.show()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training VAE...
Epoch 1/5
79/79 ━━━━━━━━━━━━━━━━━━━━ 8s 24ms/step - loss: 0.0000e+00
Epoch 2/5
79/79 ━━━━━━━━━━━━━━━━━━━━ 2s 22ms/step - loss: 0.0000e+00
Epoch 3/5
79/79 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.0000e+00
Epoch 4/5
79/79 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - loss: 0.0000e+00
Epoch 5/5
79/79 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.0000e+00
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 84ms/step
```

